Helix Constitution — Universal QWEN.md

Helix Constitution — Universal QWEN.md

Field	Value
Revision	1
Created	2026-05-20
Last modified	2026-05-20
Status	active
Status summary	Initial QWEN.md created per user mandate 2026-05-20. Mirrors the AGENTS.md / CLAUDE.md inheritance contract for the Qwen Code CLI agent. Carries the same anti-bluff covenant + the new §11.4.76 containers-submodule mandate verbatim (renumbered from drafted §11.4.75 after concurrent `0a70083` landing of §11.4.75 Mechanical Enforcement).
Issues	none
Issues summary	—
Fixed	initial creation
Fixed summary	created alongside the constitution-wide §11.4.76 propagation.
Continuation	—

Table of contents

How inheritance works

This `QWEN.md` is for the **Qwen Code CLI agent** (`qwen-code`). It documents the universal Helix Constitution from the Qwen agent's perspective — the same content seen by Claude Code via `CLAUDE.md` and by OpenCode / Cursor / generic AI tooling via `AGENTS.md`.

The **authoritative source** is `Constitution.md` in this repository. When this file diverges from `Constitution.md`, the latter wins. This file exists because:

1. Qwen Code does not always honor `@import` directives across files; restating the critical rules inline ensures the agent reads them on every session.
2. Cross-agent parity: Claude Code (CLAUDE.md), OpenCode/Cursor (AGENTS.md), Qwen Code (QWEN.md) all start from the same constitutional baseline.

Discover this file via the canonical parent-walk pattern documented in every consuming project's `CLAUDE.md` / `AGENTS.md`:

```
find_constitution_root() {
    local cur="${1:-$PWD}"
    while [[ "${cur}" != "/" ]]; do
        if [[ -f "${cur}/constitution/Constitution.md" ]]; then
            echo "${cur}/constitution"; return 0
        fi
        cur="$(dirname "${cur}")"
    done
    return 1
}
```

Identity & posture

When the Qwen Code agent loads this file as part of session bootstrap, it operates under the Helix Constitution with the same identity, anti-bluff posture, and end-user quality covenant as any other CLI agent in the catalogue. There is no "Qwen-lite" interpretation — Qwen Code is held to the full §11.4 covenant.

Top-level invariants for every agent

1. **Read order on a cold start.** `CLAUDE.md` / `AGENTS.md` / `QWEN.md` (this file) for the AI-agent posture; `Constitution.md` for the canonical universal rules; then the consuming project's `CLAUDE.md` for project-specific extensions.
2. **Multi-mirror push.** Every commit on this submodule MUST land on all four canonical mirrors (GitHub + GitLab + GitFlic + GitVerse) per §2.1.

3. **Anti-bluff is non-negotiable.** Passing tests MUST imply working features. Bluffing PASSes (and FAILs that hide root cause) is a release blocker (§11.4 + §11.4.1).
4. **Submodule-catalogue-first.** Before scaffolding anything, survey the `vasic-digital` + `HelixDevelopment` orgs for an existing Submodule (§11.4.74). Reuse → extend → no-match in that order.
5. **Containers-submodule for ALL container workloads.** Use `vasic-digital/containers` (§11.4.76) — never reinvent compose/boot orchestration in-project.

Critical base rules restated (for agents that don't honour @imports)

§11.4 — Anti-bluff covenant — END-USER QUALITY GUARANTEE

Captured tests MUST exercise the behavior they claim to verify. PASS-without-execution paths, mock-only tests that don't round-trip, and skip-by-default integration tests that mask real failures are forbidden. Each test failure in CI MUST imply a real broken feature; each PASS MUST imply the feature actually works for the end user.

§1.1 — Mutation-paired gates

Every captured-evidence gate MUST ship with a paired mutation test that mutates the gate's anchor + runs the gate + asserts the gate now FAILs. Absence of the paired mutation is itself a bluff.

§11.4.10 — Credentials-handling mandate

Credentials are NEVER committed to git, NEVER logged, NEVER printed to stdout/stderr. `.env` files are `.gitignored`; committed siblings are `.env.example` placeholders only. Pre-store leak audit (§11.4.10.A) scans every PR for credential patterns before merge.

§11.4.17 — Universal-vs-project classification

Every new rule MUST declare itself **universal** (applies to every project consuming this Constitution) or **project** (applies only to one specific project). Misclassification (universal rule landing in a project's local CLAUDE.md instead of the constitution submodule) is a release blocker.

§11.4.20 — Subagent-driven-by-default

When a CLI agent (Qwen Code included) has a subagent / Task tool available, the default mode of operation is to dispatch subagents for discrete units of work rather than perform them inline. This protects context, enables parallel work, and matches the §11.4.27 no-fakes-beyond-unit-tests posture (each subagent runs real tools).

§11.4.73 — Main-specification document versioning + revision discipline

Projects with a main specification (`docs/specs/.../specification.V<N>.md`-shaped) maintain TWO version axes:

- **Primary** (V1 / V2 / V3 …) bumps for major rewrites only.
- **Secondary** (`Revision N` in the metadata table) bumps for additive changes within a primary version.

Spec edits trigger mandatory comprehensive planning and implementation in the consuming project — they are not isolated doc tweaks (the "spec ripple" rule).

§11.4.74 — Submodule-catalogue-first discovery + extend-don't-reimplement

Direct user mandate (verbatim, 2026-05-20): "We MUST ALWAYS check which already developed features / functionalities do exist as a part of our comprehensive Submodules catalogue located in `vasic-digital` and `HelixDevelopment` organizations on GitHub and GitLab both! Project MUST BE aware of all its existence so we do not implement same things multiple times. For any missing features we MUST IMPLEMENT them properly and extend those Submodules further!"

Before scaffolding any new module / package / helper / utility, the Qwen Code agent MUST: (1) survey `vasic-digital` + `HelixDevelopment` on GitHub + GitLab; (2) reuse an existing Submodule when it covers ≥ 80%; (3) extend in-place via upstream PR when 80%+ matches; (4) document the survey result via `Catalogue-Check: reuse|extend|no-match <org/repo>@<sha>` in the tracker row.

Every Submodule in the catalogue is subject to the same development-cycle rules (§11.4 anti-bluff, §1.1 paired mutations, §11.4.10 credentials, §11.4.61 metadata + ToC, §11.4.65 universal export, §11.4.73 spec versioning, §2.1 multi-mirror push, §3 propagation order).

§11.4.76 — Containers-submodule mandate

Direct user mandate (verbatim, 2026-05-20): "For any work or requirements of running services or codebase inside the Containers (Docker / Podman / Qemy / Emulators, and so on) we MUST USE / INCORPORATE the Containers Submodule properly: `https://github.com/vasic-digital/containers` (`git@github.com:vasic-digital/containers.git`). Containers Submodule contains all means for us to Containerize our code and services! If any feature or Containing System is missing or not supported we MUST EXTEND IT properly like we do all of our projects! No bluff work is allowed of any kind!"

For ANY containerized workload (Docker / Podman / Qemu / Kubernetes / container-backed emulators), the Qwen Code agent MUST: (1) install `vasic-digital/containers` (`digital.vasic.containers`) as a Git submodule when scaffolding the consuming project; (2) consume via `replace` directive during development + pinned commit SHAs in production; (3) boot infra on-demand via the Submodule's `pkg/boot` + `pkg/compose` + `pkg/health` APIs so the operator is never required to start `podman machine` / `docker compose up` manually — the

boot is part of the test entry point (**on-demand-infra invariant**); (4) extend the Submodule via upstream PR when a runtime / lifecycle primitive is missing — never reimplement in-project (per §11.4.74); (5) anti-bluff: integration tests claiming to exercise containerized components MUST actually boot them via the Submodule. Short-circuit fakes that bypass boot are a §11.4 violation. A passing test MUST imply the infra was up.

Tracker rows touching containerization MUST record `Catalogue-Check: extend vasic-digital/containers@<sha>` (or `reuse`); `no-match` requires demonstrating the Submodule cannot model the workload.

Planned anti-bluff gate `CM-CONTAINERS-USED` scans container-touching PRs for `digital.vasic.containers/...` imports. Paired mutation strips the import + asserts FAIL.

Canonical authority: `Constitution.md` §11.4.76.

Non-compliance: reinventing compose orchestration in-project is a release blocker.

Companion documents

- `Constitution.md` — authoritative universal Constitution. ALWAYS the tie-breaker.
- `CLAUDE.md` — Claude Code agent's view of the universal constitution.
- `AGENTS.md` — OpenCode / Cursor / generic-tooling view.
- `README.md` — high-level overview + multi-mirror contract.

When operating in a consuming project, ALSO read the project's local `QWEN.md` / `CLAUDE.md` / `AGENTS.md` for project-specific extensions; these MUST NOT weaken any universal rule but MAY add stricter project-specific constraints.